# Paper 06 - Causal Code

CQE-paper-06
CQECMPLX Formal Paper Series
Review-facing PDF built 2026-06-12

## Construction Basis

Cast every dependency between objects, proofs, tools, and papers as a typed causal edge.

This PDF is the clean paper artifact. Source extracts, kernels, receipts, tool notes, workbook material, and package bindings are used as construction evidence; they are not treated as separate review-facing papers.

## Evidence Inputs

- promoted formal paper: production\formal-papers\CQE-paper-06\FORMAL_PAPER.md
- paper kernel manifest: production\paper-kernels\papers\CQE-paper-06\paper_kernel_manifest.json
- verifier: production\formal-papers\CQE-paper-06\verify_causal_code.py
- receipt: production\formal-papers\CQE-paper-06\causal_code_receipt.json (Typed causal edge contract: pass)
- body: production\papers\CQE-paper-06\PAPER-BODY.md
- source: production\papers\CQE-paper-06\SOURCE.md
- formal: production\papers\CQE-paper-06\01-CQE-formal\FORMAL.md
- tool: production\papers\CQE-paper-06\02-CQE-tool\TOOL.md
- workbook: production\papers\CQE-paper-06\03-CQE-workbook\WORKBOOK.md
- recovered_output: production\lib-forge\recovered\papers_output\CQE-paper-06.md

# Paper 06 - Causal Code

## Abstract

Paper 06 defines the causal code used by the paper suite: every dependency between papers, proofs, tools, receipts, and obligations must be represented as a typed edge. The goal is not to declare the whole corpus complete. The goal is to make incompleteness traceable.

The polished claim is a schema theorem. A causal edge is valid only when it has a source, target, edge type, receipt, and status. A causal graph is usable only when its active proof dependencies are acyclic or when cycles are explicitly typed as review/feedback loops rather than hidden proof dependencies.

## Proof/Exposure Hierarchy

The proof-carrying content of this paper is the mathematics: the definitions, lemmas, constructions, examples, and receipts that establish the claimed result. Paper 00, hand routes, analog tools, workbook language, and obligation ledgers are supplemental validation and exposure layers. They exist to make the math inspectable, reproducible, and accessible without requiring a particular software stack. The hand route is not the purpose of the paper; it is a way to expose the same state transitions with ordinary marks, tokens, lines, or any equivalent physical substitute.

## Definitions

A **causal vertex** is a paper, proof, tool, receipt, obligation, or product artifact.

A **causal edge** is a record:

```
source
target
edge_type
receipt
status
```

Allowed edge types are:

```
uses
proves
refines
obligates
transports
repairs
constrains
verifies
```

Allowed statuses are:

```
open
closed
deferred
rejected
```

An edge with status `open` is not a proof closure. It is a tracked obligation.

## Main Claim

**Theorem 6.1, Typed Causal Edge Contract.** A paper-kernel dependency is admissible to the CQECMPLX production graph only if it is represented by a typed causal edge with source, target, edge type, receipt, and status. Active proof dependencies must be acyclic unless the cycle is explicitly typed as review, feedback, or obligation routing rather than proof support.

### Proof

Without a source and target, a dependency cannot be located. Without an edge type, the dependency cannot be interpreted. Without a receipt, it cannot be replayed. Without status, the graph cannot distinguish proof closure from open obligation. Therefore all five fields are necessary for a production causal edge.

If a proof dependency cycle is hidden, then a claim can support itself through the graph. That is not proof; it is circular support. Requiring active proof dependencies to be acyclic prevents this. Cycles are still allowed as explicit review or feedback routes, but then their type prevents them from masquerading as proof closure. QED.

## Concrete Graph From Papers 01-05

The first polished chain gives a small causal graph:

```
Paper 01 uses Paper 00 contract.
Paper 02 uses Paper 01 LCR carrier.
Paper 03 uses Paper 01 LCR carrier and Paper 02 correction coordinates.
Paper 04 consumes Paper 02 residue and Paper 03 coordinates.
Paper 05 carries Paper 04 constraints.
Paper 06 formalizes the typed edge contract for all of the above.
```

This graph is intentionally partial. It proves that the schema can track polished paper dependencies; it does not claim that every obligation in all 32 papers is already resolved.

## Falsifiers

The verifier must reject:

```
1. An edge with no receipt.
2. An edge with an unknown type.
3. A proof cycle disguised as closure.
4. A graph that labels open obligations as resolved.
```

These falsifiers protect the suite from becoming a pile of agreeable prose.

## Hand Reconstruction

- Write each paper or tool as a vertex.
- Draw an arrow for every dependency.
- Label the arrow with one allowed edge type.
- Attach a receipt id.
- Mark the edge status.
- Trace the proof-support subgraph and check for hidden cycles.
- Keep open obligations as open.

## Code Reconstruction

The production verifier is:

```
production/formal-papers/CQE-paper-06/verify_causal_code.py
```

It verifies:

```
1. Every edge has required fields.
2. Every edge uses an allowed type and status.
3. The polished Papers 01-06 graph is acyclic for proof-support edges.
4. Open obligations remain open.
5. Invalid edges and hidden proof cycles are rejected.
```

## Validation and Hidden-Guess Layer

Useful hidden-guess prompts:

```
What fields are required on a causal edge?
Can an open obligation be counted as resolved?
Can a proof-support graph contain a hidden cycle?
What edge type connects Paper 04 to Paper 05?
```

Expected answers:

```
source, target, type, receipt, status
no
no
transports or constrains, depending on the specific route
```

## Open Obligations

- Wire `verify_causal_code` into `cqe_engine.formal`.
- Populate the full 32-paper graph from all formal receipts.
- Decide which cycles are allowed as review loops and which are rejected as

proof cycles.

- Replace placeholder receipt ids with repository-stable artifact hashes.

## Conclusion

Causal code is the production graph discipline for the suite. It makes every dependency explicit and prevents open obligations from being mistaken for closed proof.